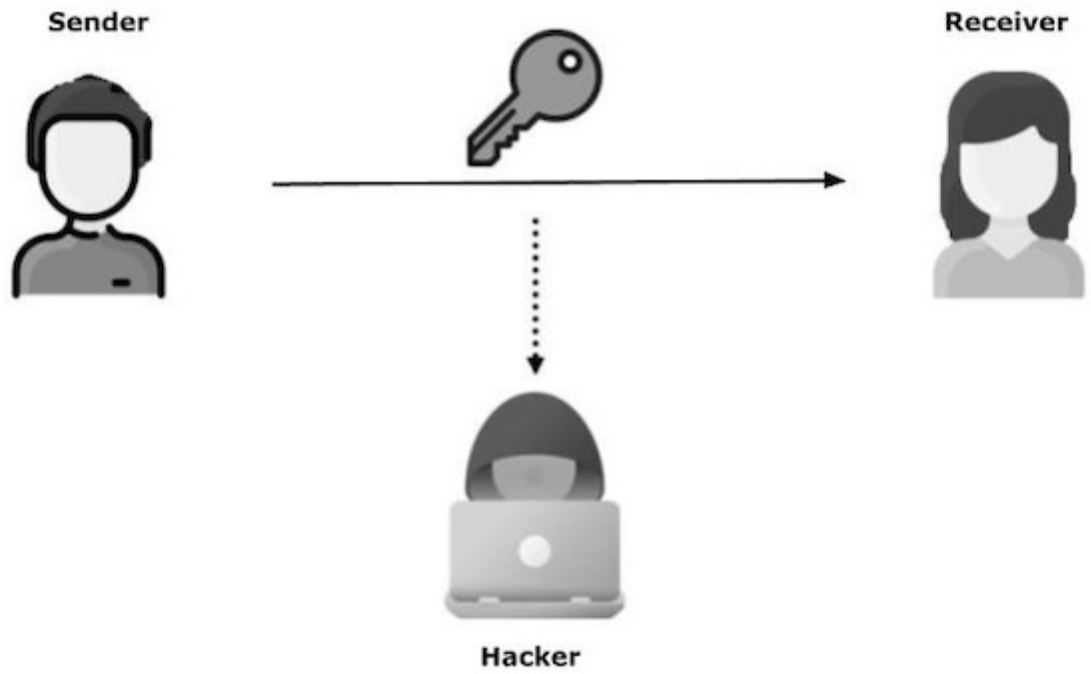
Cryptography - Diffie-Hellman Algorithm

Diffie-Hellman key exchange is a type of digital encryption in which two different parties securely exchange cryptographic keys over a public channel without their communication being sent over the internet. Both parties use symmetric cryptography to encrypt and decrypt their messages.

Whitfield Diffie and Martin Hellman published it in 1976 as one of the earliest practical cases of public key cryptography.

The Diffie-Hellman key exchange raises numbers to a specific power to generate decryption keys. The components of the keys are never directly transferred, making the task of a potential code breaker mathematically impossible. The approach does not share any information during the key exchange. Despite having no prior knowledge of each other, the two parties collaborate to produce a key.



History of Diffie-Hellman

Whitfield Diffie and Martin Hellman developed the Diffie-Hellman algorithm in 1976, which opened the path for public-key cryptography and greatly improved digital security. It was developed as a solution for secure key exchange over insecure channels, substituting symmetric key distribution methods and providing the groundwork for secure communication protocols such as SSL/TLS.

The algorithm has had a considerable impact on world geopolitics, with one example being its usage to secure top-secret communications between NATO countries during the late Cold War era.

How Diffie-Hellman Works?

To use Diffie-Hellman, two end users, Alice and Bob, must agree on positive whole integers p and q, where p is a prime number and q is a generator of p. The generator q is a number that, when raised to positive whole-number powers smaller than p, never gives the same outcome for any two such whole numbers. The value of p can be big, while the value of q is typically small.

After Alice and Bob have decided on p and q in secret, they select positive whole-number personal keys a and b. Both are less than the prime number modulus, p. Neither user shares their personal key with anybody; ideally, they remember these numbers and do not write them down or save them anywhere. After that, Alice and Bob compute public keys a* and b* based on their personal keys using the algorithms below −

$a^* = q^a \bmod p$

$b^* = q^b \bmod p$

It is possible for the two users to exchange their public keys, a^* and b^*, throughout an insecure communications channel, like the internet or a company's wide area network. Using their individual personal keys, each user can generate a number x from these public keys. Alice uses the following formula to compute x −

$x = (b^*) \bmod p$

Bob uses the following formula to compute x: $x = (a^*) \bmod p$

Either of the two formulas above gives the same result for the value of x. But the private keys a and b, which are necessary for calculating x, have not been sent via a public channel. Even with the help of an efficient system capable of doing millions of attempts, a potential hacker has nearly no chance of properly determine x due to its huge size and seemingly random nature. So, using the decryption key x, the two users can theoretically have private conversations over a public channel using any encryption technique they choose.

Step by Step Guide

Let us use the simple case of Alice and Bob, two friends, to describe Diffie-Hellman −

Selecting the Prime Number and Generator

Bob and Alice have decided on a prime number, p = 23.

Also, they are also agree on a generator number, g = 5.

Selecting Private Keys

Alice selects a secret number, for example, a = 6.

Bob selects a secret number, for example, b = 15.

Determine the public keys

Alice first computes A = g^a mod p, which gives A = 5^6 mod 23, or 8.

Bob uses the formula B = g^b mod p to get B = 5^{15} mod 23, or 19.

Exchange of Public Keys

Alice provides Bob her computed public key, A = 8.

Bob sends Alice his calculated public key, B = 19.

Compute Shared Key

Alice uses Bob's public key to compute the shared key −

$K_A = B^a$ mod p −

$K_A = 19^6$ mod 23, which is 2.

Bob uses Alice's public key to calculate the shared key −

$K_B = A^b$ mod p −

$K_B = 8^{15}$ mod 23, which is 2.

As a result, Alice and Bob can now encrypt their messages using the same shared key, K = 2.

It is very difficult for someone to figure out the shared key even if they manage to eavesdrop and know the prime number p=23 and the generator g=5. This is because they do not know either Alice's or Bob's secret number.

Implementation of Diffie-Hellman

We will implement the Diffie-Hellman algorithm using Python, Java and C++.

Implement using Python

The Diffie-Hellman key exchange algorithm is demonstrated in the below Python code, providing secure communication between Alice and Bob.

In order to avoid overflow errors, the calculate_power function effectively computes a^b mod P, where a, b, and P are large numbers. Important parameters that are agreed upon by both sides are initialised, like the prime number (prime) and generator value (generator). Each side selects their private key (private_key_bob and private_key_alice), and then uses modular exponentiation to calculate their public key. In the end, secure

communication is made possible when Alice and Bob separately compute the shared secret key (secret_key_alice and secret_key_bob) following the exchange of public keys.

Following is the Diffie-Hellman implementation using Python −

Example

```
def calculate_power(base, exponent, modulus):
  """Power function to return value of (base ^ exponent) mod modulus."""
  if exponent == 1:
    return base
  else:
    return pow(base, exponent) % modulus



if __name__ == "__main__":
  prime = 23
  generator = 5
  private_key_alice = 6
  private_key_bob = 15


  x = calculate_power(generator, private_key_alice, prime)
  y = calculate_power(generator, private_key_bob, prime)


  secret_key_alice = calculate_power(y, private_key_alice, prime)
  secret_key_bob = calculate_power(x, private_key_bob, prime)


  print("Prime number (P):", prime)
  print("Generator value (G):", generator)
  print("Alice's private key (a):", private_key_alice)
  print("Bob's private key (b):", private_key_bob)
  print("Secret key for Alice:", secret_key_alice)
  print("Secret key for Bob:", secret_key_bob)
```

Following is the output of the above example −

Input/Output

Prime number (P): 23
Generator value (G): 5
Alice's private key (a): 6
Bob's private key (b): 15
Secret key for Alice: 2
Secret key for Bob: 2

Implement using C++

The code makes it possible for Alice and Bob to communicate securely. The method effectively computes the shared secret key by modular exponentiation, providing compatibility with large integers. Alice and Bob then produce their private keys and calculate their corresponding public keys. Each party separately calculates the shared secret key by using these public keys along with their own private keys.

The below code implements the Diffie-Hellman key exchange algorithm in C++.

Example

```
#include <iostream>
#include <cmath>

using namespace std;


long long int calculatePower(long long int base, long long int exponent, long long int modulus) {
  if (exponent == 1)
    return base;
  else
    return (((long long int)pow(base, exponent)) % modulus);
}

int main() {
  long long int prime, generator, secretAlice, secretBob, publicAlice, publicBob, secretKeyAlice,
secretKeyBob;


  prime = 23;
  cout << "Prime number (P): " << prime << endl;

  generator = 5;
  cout << "Generator value (G): " << generator << endl;
```

```
   secretAlice = 6;
   cout << "Alice's private key (a): " << secretAlice << endl;

   publicAlice = calculatePower(generator, secretAlice, prime);

   secretBob = 15;
   cout << "Bob's private key (b): " << secretBob << endl;

   publicBob = calculatePower(generator, secretBob, prime);

   secretKeyAlice = calculatePower(publicBob, secretAlice, prime);
   secretKeyBob = calculatePower(publicAlice, secretBob, prime);

   cout << "Secret key for Alice: " << secretKeyAlice << endl;
   cout << "Secret key for Bob: " << secretKeyBob << endl;

   return 0;
}
```

Following is the output of the above example −

Input/Output

```
Prime number (P): 23
Generator value (G): 5
Alice's private key (a): 6
Bob's private key (b): 15
Secret key for Alice: 2
Secret key for Bob: 2
```

Implement using Java

So in this implementation we will use same approach we have used in the C++ program. But in this we have created a class called DiffieHellman inside this we have calculatePower() function and main() function. The

implementation using Java is as follows −

Example

```
public class DiffieHellman {


  private static long calculatePower(long base, long exponent, long modulus) {
    if (exponent == 1)
      return base;
    else
      return (((long) Math.pow(base, exponent)) % modulus);
  }


  public static void main(String[] args) {
    long prime, generator, x, privateKeyAlice, y, privateKeyBob, secretKeyAlice, secretKeyBob;




    prime = 23;
    System.out.println("Prime number (P): " + prime);


    generator = 5;
    System.out.println("Generator value (G): " + generator);


    privateKeyAlice = 6;
    System.out.println("Alice's private key (a): " + privateKeyAlice);


    x = calculatePower(generator, privateKeyAlice, prime);


    privateKeyBob = 15;
    System.out.println("Bob's private key (b): " + privateKeyBob);
```

```
    y = calculatePower(generator, privateKeyBob, prime);


    secretKeyAlice = calculatePower(y, privateKeyAlice, prime);
    secretKeyBob = calculatePower(x, privateKeyBob, prime);

    System.out.println("Secret key for Alice: " + secretKeyAlice);
    System.out.println("Secret key for Bob: " + secretKeyBob);
  }
}
```

Following is the output of the above example −

Input/Output

```
Prime number (P): 23
Generator value (G): 5
Alice's private key (a): 6
Bob's private key (b): 15
Secret key for Alice: 2
Secret key for Bob: 2
```

Applications/Where is it used?

The development of a secure channel for key generation and sharing for symmetric key algorithms is the goal of the Diffie-Hellman key exchange. It is frequently used for forward security, encryption, and password-authenticated key agreement. Man-in-the-middle attacks (MitM) are prevented by using password-authenticated key agreements. Forward secrecy processes protect against key loss by producing new key pairs for each session.

Diffie-Hellman key exchange is used in several security protocols, including Transport Layer Security (TLS), Secure Shell (SSH), and IP Security (IPsec). For example, in IPsec, the encryption mechanism is used to generate and rotate keys.

While the Diffie-Hellman key exchange can be used to create both public and private keys, the Rivest-Shamir-Adleman algorithm, or RSA algorithm, is another option because it can sign public key certificates.

Example of Diffie-Hellman Key Exchange

The Diffie-Hellman key exchange technique is an encryption technique. For example, It can be used by Alice and Bob to exchange sensitive data over an open public network while protecting themselves from hackers and eavesdroppers. This open public network can be identified at a cafe.

A function is run on the secret keys that Alice and Bob privately choose in order to generate the public key.

The results are shared, not the function. It will be challenging to figure out what function the numbers came from if someone else is listening in, even if they have access to all the numbers involved.

After that, Alice and Bob execute separate functions by using the outcomes that they received from the other party, their unique secret number, and the initial prime value. After then, Alice and Bob find out a shared secret key that nobody else is able to find out. Bob and Alice can now talk to each other without worrying about outsiders.

Vulnerabilities

The main constraints of Diffie-Hellman in its basic version is the lack of authentication. Communications using Diffie-Hellman only are vulnerable to MitM. Diffie-Hellman should be used in conjunction with a recognised authentication method, like digital signatures, to verify user identities across a public communication medium.

The Diffie-Hellman key exchange is also vulnerable to logjam attacks, particularly against the TLS protocol. Logjam attacks reduce TLS connections to 512-bit cryptography, allowing an attacker to access and change data transmitted across the connection. Diffie-Hellman key exchange can still be secure if done properly. Logjam attacks, for example, will fail when using a 2,048-bit key.

Advantages/Benifits

It is an effective key exchange and secure connection setup without needing prior preparation.

It provides forward secrecy, which makes sure that previous conversations are safe even in a situation that one key is compromised.

The lack of a requirement for secure secret key transmission lowers the possibility of key compromise while in transit.

This technique enables a large number of users to securely interact, making it suitable for usage in large-scale applications.

Disadvantages/Limitations

Needs a secure first public key exchange to prevent man-in-the-middle and eavesdropping attacks.

Vulnerable to brute-force attacks from adversaries possessing powerful computing capabilities.

Highly computational, which may have an effect on performance, particularly for big prime numbers.

Lacks authentication, demanding further steps to confirm the parties' identities when communicating.

Differences between Diffie - Hellman and RSA

The differences are in the following table −

Basis of Differences	Diffie-Hellman	RSA
Key Functionality	In this algorithm, the same key is used by both the transmitter and receiver.	RSA broadly utilizes a cryptographic connection as it follows an encrypted technique
Key Generation	Both sides generate their keys.	Both the public and private keys are used for security.
Performance	It is efficient for key exchange, but slower for encryption/decryption	It is fast for encryption/decryption but slow for key exchange.
Key Length	Longer key lengths are usually required to achieve the same level of security	Shorter key lengths allow Diffie-Hellman to provide the same level of security as longer keys.
Usage	It is commonly used for secure key exchange in symmetric encryption systems	It is used for various purposes for securing by encrypting and decrypting data.